

Nhóm 12

Dương Hoàng Việt - 24520036

Hồ Hữu Tây - 24520029

1 Subtask 1: $N, M \leq 10^3$

1.1 Phương pháp thiết kế thuật toán

Với kích thước dữ liệu nhỏ ($N, M \leq 1000$), chúng tôi sử dụng phương pháp **Vét cạn (Brute Force)**. Ý tưởng chính là duyệt qua từng Hunter, và với mỗi Hunter, ta kiểm tra tất cả N vùng Aura để tìm ra vùng nhỏ nhất (có diện tích nhỏ nhất hoặc cấp độ sâu nhất) chứa Hunter đó.

Quy trình thực hiện cho mỗi Hunter tại vị trí (x_H, y_H) :

1. Khởi tạo biến `best_zone` là rỗng (hoặc đại diện cho vùng ngoài cùng).
2. Duyệt qua từng vùng i từ 1 đến N với tọa độ $(x_{1i}, y_{1i}, x_{2i}, y_{2i})$.
3. Kiểm tra điều kiện bao hàm: Nếu $x_{1i} < x_H < x_{2i}$ và $y_{1i} < y_H < y_{2i}$:
 - Nếu `best_zone` chưa có giá trị hoặc vùng i nằm bên trong `best_zone` (dựa trên tính chất lồng nhau của đề bài, vùng nào có x_1 lớn hơn sẽ nhỏ hơn), ta cập nhật `best_zone = i`.
4. Cộng điểm Aura của Hunter vào tổng Aura của `best_zone` tìm được.

1.2 Tại sao phương pháp này phù hợp?

- **Tính đơn giản:** Thuật toán cài đặt rất nhanh, ít xảy ra lỗi logic, phù hợp để kiểm tra tính đúng đắn (test) cho các giải thuật phức tạp hơn.
- **Giới hạn dữ liệu:** Với $N, M \leq 10^3$, tổng số phép tính là khoảng $N \times M = 10^6$ phép tính. Máy tính hiện đại có thể xử lý 10^8 phép tính mỗi giây, do đó thuật toán chạy trong khoảng 0.01s - 0.1s, hoàn toàn thỏa mãn giới hạn thời gian.

1.3 Phân tích độ phức tạp

Độ phức tạp thời gian: $\Theta(N \cdot M)$

Thuật toán sử dụng cấu trúc hai vòng lặp lồng nhau:

- Vòng lặp ngoài duyệt qua danh sách M Hunter.
- Vòng lặp trong duyệt qua toàn bộ N vùng Aura.

Trong mọi trường hợp (tốt nhất hay xấu nhất), thuật toán luôn thực hiện chính xác N phép so sánh cho mỗi Hunter mà không có cơ chế dừng sớm (early termination). Do đó, tổng số phép toán luôn là một hằng số nhân với $N \cdot M$.

$$T(N, M) = \Theta(N \cdot M)$$

Độ phức tạp không gian: $\Theta(N + M)$

Thuật toán chỉ sử dụng bộ nhớ để lưu trữ dữ liệu đầu vào (mảng chứa N vùng và mảng chứa M Hunter). Các biến phụ trợ (như `best_zone`) chỉ tốn bộ nhớ hằng số $O(1)$. Do đó, độ phức tạp không gian tuyến tính theo kích thước đầu vào.

2 Subtask 2: $W, H \leq 10^3$

2.1 Phương pháp thiết kế thuật toán

Ở subtask này, kích thước không gian của bản đồ khá nhỏ ($W, H \leq 1000$), trong khi số lượng Hunter và Vùng có thể lớn. Chúng tôi sử dụng phương pháp **Mô phỏng trên lưới 2 chiều (2D Grid Simulation)**.

Cụ thể, chúng ta coi bản đồ là một ma trận 2 chiều `Map[W][H]`. Mỗi phần tử `Map[x][y]` sẽ lưu trữ ID của vùng Aura nhỏ nhất bao chứa tọa độ đó. Quy trình thực hiện gồm 2 giai đoạn:

1. Tiền xử lý (Pre-computation):

- Sắp xếp N vùng Aura theo diện tích giảm dần.
- Duyệt qua từng vùng và "tô màu" lên mảng `Map`: Với mỗi vùng k , ta gán `Map[x][y] = k` cho tất cả các điểm (x, y) nằm trong vùng đó.
- Nhờ việc tô từ vùng lớn đến vùng nhỏ, các vùng nhỏ hơn (nằm trong) sẽ ghi đè lên giá trị của vùng lớn hơn (bao ngoài), đảm bảo tại mỗi ô nhớ luôn lưu giữ ID của vùng nhỏ nhất (trong cùng nhất).

2. Truy vấn (Query):

Với mỗi Hunter, ta chỉ cần truy xuất giá trị `Map[x][y]` để biết kết quả ngay lập tức.

2.2 Tại sao phương pháp này phù hợp?

Phương pháp này phù hợp dựa trên ba lập luận chính sau:

- Chiến lược đánh đổi không gian lấy thời gian (Space-Time Trade-off):** Đặc thù của Subtask 2 là không gian tọa độ bị giới hạn chặt chẽ (1000×1000). Bằng cách chấp nhận tốn bộ nhớ để lưu toàn bộ bản đồ, ta chuyển độ phức tạp của việc tìm kiếm từ phụ thuộc vào số lượng vùng (N) về hằng số ($O(1)$). Điều này đặc biệt hiệu quả khi số lượng truy vấn Hunter (M) rất lớn.
- Khả thi về mặt bộ nhớ:** Mảng 2 chiều kích thước 1000×1000 chứa 10^6 số nguyên (integer). Với kích thước tiêu chuẩn 4 bytes cho một số nguyên, mảng này chỉ chiếm khoảng:

$$10^6 \times 4 \text{ bytes} \approx 4 \text{ MB}$$

Con số này rất nhỏ so với giới hạn bộ nhớ thông thường của các hệ thống chấm bài (thường là 256MB hoặc 512MB), do đó giải pháp này hoàn toàn an toàn về mặt không gian lưu trữ.

- Đơn giản hoá bài toán hình học:** Thay vì phải xử lý các công thức hình học phức tạp để kiểm tra bao hàm hay giao nhau, phương pháp này chuyển bài toán về các thao tác mảng cơ bản (gán và truy xuất). Điều này giúp giảm thiểu rủi ro sai sót trong cài đặt và xử lý các trường hợp biên (edge cases).

2.3 Phân tích độ phức tạp

Độ phức tạp thời gian: $\Theta(\sum S_i + M)$

Thời gian thực thi được chia làm hai phần:

- **Giai đoạn Tiền xử lý:** Ta phải duyệt qua từng ô của từng vùng Aura để gán giá trị. Gọi S_i là diện tích (số lượng điểm nguyên) của vùng thứ i . Tổng thời gian để xây dựng bản đồ là tổng diện tích của tất cả các vùng phủ lên nhau: $\Theta(\sum_{i=1}^N S_i)$. Trong trường hợp xấu nhất (tất cả các vùng đều có kích thước xấp xỉ $W \times H$), độ phức tạp là $\Theta(N \cdot W \cdot H)$.
- **Giai đoạn Truy vấn:** Mỗi Hunter chỉ cần một thao tác truy cập mảng để xác định vùng. Thời gian cho M Hunter là $\Theta(M)$.

Tổng quát: $T(W, H, N, M) = \Theta(\sum S_i + M)$.

Độ phức tạp không gian: $\Theta(W \cdot H)$

Không gian lưu trữ bị chi phối hoàn toàn bởi kích thước của mảng bản đồ 2 chiều $\text{Map}[W][H]$. Các biến phụ trợ khác không đáng kể. Do đó, độ phức tạp không gian là hằng số đối với N và M , và bậc hai đối với kích thước cạnh bản đồ.

$$S(W, H) = \Theta(W \cdot H)$$

3 Subtask 3: Không ràng buộc ($N, M \leq 10^5, W, H \leq 10^9$)

3.1 Phương pháp thiết kế thuật toán

Đây là bài toán tổng quát với tọa độ rất lớn và số lượng đối tượng nhiều. Chúng tôi kết hợp ba kỹ thuật nâng cao để giải quyết: **Nén tọa độ**, **Quét dòng (Sweep Line)** và **Cây phân đoạn (Segment Tree)**.

Quy trình thực hiện như sau:

1. **Nén tọa độ (Coordinate Compression):** Vì tọa độ Y lên tới 10^9 nhưng chỉ có tối đa $2N + M$ giá trị quan trọng, ta ánh xạ các giá trị Y này về miền giá trị liên tục $[0, K]$ (với $K \approx 3 \cdot 10^5$) để làm chỉ số cho cây Segment Tree.
2. **Xử lý sự kiện Quét dòng:** Biến đổi bài toán 2D thành bài toán 1D động bằng cách quét một đường thẳng dọc trục X . Các sự kiện được sắp xếp tăng dần theo tọa độ X :
 - *Sự kiện Start (Vào vùng):* Cập nhật đoạn $[y_1, y_2]$ trên Segment Tree với ID của vùng mới. Trước khi cập nhật, ta lưu lại ID của vùng "cha" (vùng đang bao trùm khu vực này) để khôi phục sau này.
 - *Sự kiện Hunter (Truy vấn):* Truy vấn tại điểm y_H trên Segment Tree để xác định Hunter đang đứng trong vùng nào.
 - *Sự kiện End (Ra vùng):* Cập nhật lại đoạn $[y_1, y_2]$ bằng ID của vùng "cha" đã lưu, trả lại trạng thái cũ cho khu vực này.
3. **Cây phân đoạn tối ưu (Iterative Segment Tree):** Sử dụng Segment Tree phiên bản khử đệ quy kết hợp kỹ thuật **Timestamp (Dấu thời gian)**. Thay vì dùng Lazy Propagation phức tạp, ta lưu thời điểm cập nhật tại mỗi nút. Khi truy vấn tại một điểm lá, ta đi ngược từ lá lên gốc và chọn giá trị có timestamp mới nhất (Last Update Wins).

3.2 Tại sao phương pháp này phù hợp?

Việc lựa chọn phương pháp này dựa trên sự phân tích các hạn chế của hai phương pháp trước:

- **Khắc phục hạn chế về Bộ nhớ của Subtask 2:** Với $W, H = 10^9$, ta không thể khởi tạo mảng 2 chiều (cần 10^{18} phần tử). Kỹ thuật *Nén toạ độ* giúp đưa không gian bài toán về phụ thuộc vào số lượng đối tượng (N, M) thay vì toạ độ hình học, giảm kích thước không gian lưu trữ xuống tuyến tính.
- **Khắc phục hạn chế về Thời gian của Subtask 1:** Với $N, M = 10^5$, giải thuật $O(N \cdot M)$ sẽ tốn 10^{10} phép tính (quá thời gian cho phép). Sự kết hợp giữa *Sweep Line* (giảm 1 chiều dữ liệu) và *Segment Tree* (cấu trúc dữ liệu logarit) giúp giảm độ phức tạp xuống $O((N + M) \log(N + M))$, đảm bảo xử lý mượt mà 300.000 sự kiện trong chưa đầy 1 giây.
- **Tối ưu hoá cho ngôn ngữ Python:** Việc sử dụng Segment Tree phiên bản *Iterative* (vòng lặp) thay vì *Recursive* (đệ quy) là cực kỳ quan trọng khi cài đặt bằng Python, giúp loại bỏ hoàn toàn chi phí gọi hàm (overhead) và rủi ro tràn bộ nhớ ngăn xếp (Stack Overflow).

3.3 Phân tích độ phức tạp

Độ phức tạp thời gian: $\Theta(K \log K)$

Gọi $K = N + M$ là tổng số lượng đối tượng đầu vào.

- **Sắp xếp:** Việc sắp xếp các toạ độ để nén và sắp xếp các sự kiện để quét dòng chiếm $\Theta(K \log K)$.
- **Xử lý sự kiện:** Ta có tổng cộng khoảng $2N + M$ sự kiện. Mỗi sự kiện thực hiện thao tác Update hoặc Query trên Segment Tree với chi phí $\Theta(\log(\text{size of tree}))$. Vì kích thước cây tỉ lệ với K , chi phí này là $\Theta(K \log K)$.

Tổng quát, độ phức tạp thời gian bị chi phối bởi thao tác sắp xếp và cấu trúc cây:

$$T(N, M) = \Theta((N + M) \log(N + M))$$

Độ phức tạp không gian: $\Theta(N + M)$

Không gian lưu trữ bao gồm:

- Danh sách sự kiện và mảng nén toạ độ: $\Theta(N + M)$.
- Cây Segment Tree: Kích thước cây gấp 2 hoặc 4 lần số lượng toạ độ rời rạc, tức là $\Theta(N + M)$.

Vậy thuật toán tiêu tốn bộ nhớ tuyến tính theo số lượng đầu vào.

3.4 Cài đặt thuật toán (Python)

Dưới đây là mã nguồn Python sử dụng Segment Tree khử đệ quy đã được tối ưu hoá (Accepted):

```

1  import sys
2
3  # Sử dụng buffer đọc nhanh nhất có thể
4  input = sys.stdin.readline
5
6  def solve():
7      # Đọc toàn bộ dữ liệu
8      data = sys.stdin.read().split()
9      if not data: return
10     iterator = iter(data)
11
12     try:
13         W = int(next(iterator)); H = int(next(iterator))
14         n = int(next(iterator)); m = int(next(iterator))
15     except StopIteration: return
16
17     events = []
18     y_coords = set()
19     zones_info = {}
20
21     # Đọc Zones
22     for i in range(1, n + 1):
23         x1 = int(next(iterator)); y1 = int(next(iterator))
24         x2 = int(next(iterator)); y2 = int(next(iterator))
25         events.append((x1, 0, y1, y2, i)) # Start
26         events.append((x2, 2, y1, y2, i)) # End
27         zones_info[i] = (y1, y2)
28         y_coords.add(y1); y_coords.add(y2)
29
30     # Đọc Hunters
31     for j in range(m):
32         hx = int(next(iterator)); hy = int(next(iterator))
33         haura = int(next(iterator))
34         events.append((hx, 1, hy, hy, haura)) # Query
35         y_coords.add(hy)
36
37     # Nén tọa độ
38     sorted_y = sorted(list(y_coords))
39     y_map = {val: i for i, val in enumerate(sorted_y)}
40     cnt_y = len(sorted_y)
41     events.sort(key=lambda e: (e[0], e[1]))
42
43     # Iterative Segment Tree setup
44     size = 1

```

```

45     while size <= cnt_y: size *= 2
46     tree_time = [-1] * (2 * size)
47     tree_val = [0] * (2 * size)
48     current_time = 0
49
50     def update(l, r, val):
51         nonlocal current_time
52         current_time += 1
53         l += size; r += size
54         while l <= r:
55             if l % 2 == 1:
56                 tree_time[l] = current_time; tree_val[l] = val; l += 1
57             if r % 2 == 0:
58                 tree_time[r] = current_time; tree_val[r] = val; r -= 1
59             l //= 2; r //= 2
60
61     def query(p):
62         p += size
63         res_time = -1; res_val = 0
64         while p > 0:
65             if tree_time[p] > res_time:
66                 res_time = tree_time[p]; res_val = tree_val[p]
67             p //= 2
68         return res_val
69
70     # Xử lý sự kiện
71     total_aura = [0] * (n + 1)
72     parent_map = {}
73
74     for ev in events:
75         etype = ev[1]
76         if etype == 0: # Start Rect
77             _, _, y1, y2, zid = ev
78             y1_idx = y_map[y1]; y2_idx = y_map[y2]
79             parent_id = query(y1_idx)
80             parent_map[zid] = parent_id
81             update(y1_idx, y2_idx, zid)
82         elif etype == 1: # Hunter
83             _, _, hy, _, haura = ev
84             hy_idx = y_map[hy]
85             zone_id = query(hy_idx)
86             total_aura[zone_id] += haura
87         elif etype == 2: # End Rect
88             _, _, y1, y2, zid = ev
89             y1_idx = y_map[y1]; y2_idx = y_map[y2]
90             pid = parent_map[zid]
91             update(y1_idx, y2_idx, pid)

```

```
92
93     print(*(total_aura[1:] + [total_aura[0]]))
94
95 if __name__ == '__main__':
96     solve()
```